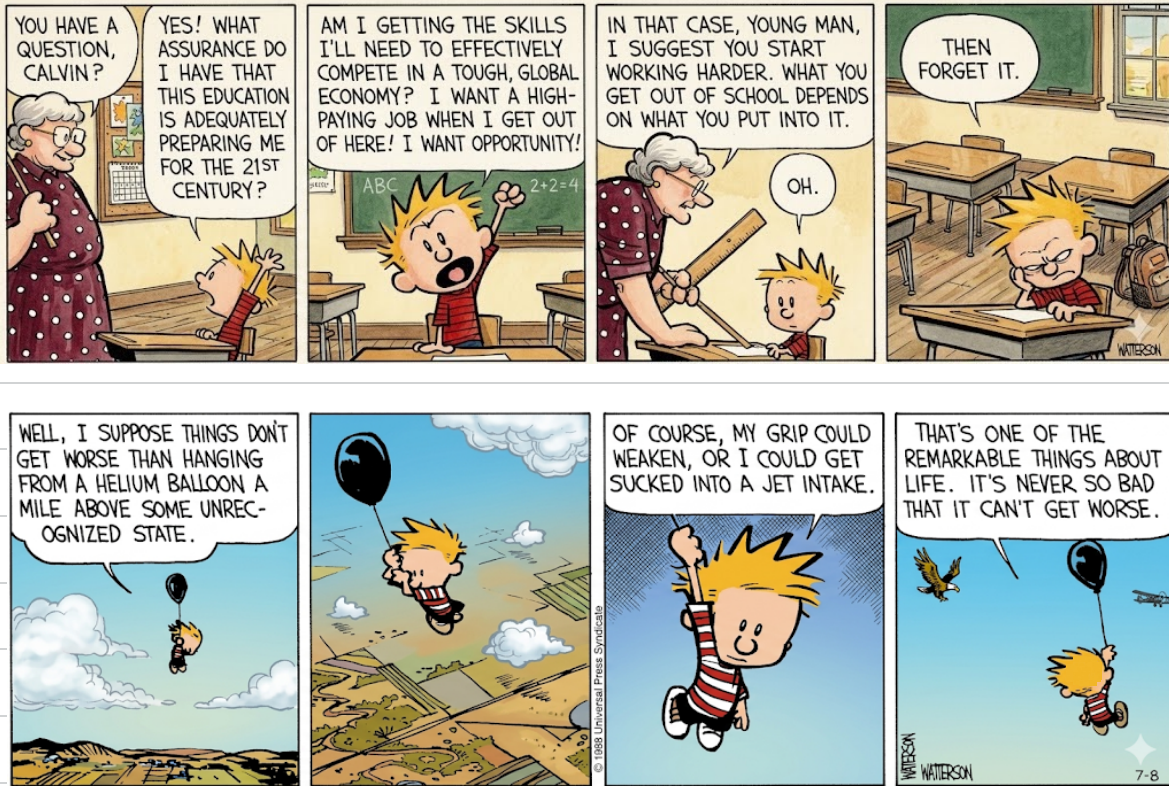


REMINDER :)



Agenda

1. Basics of ML
2. Linear Model and Gradient Descent Intuition

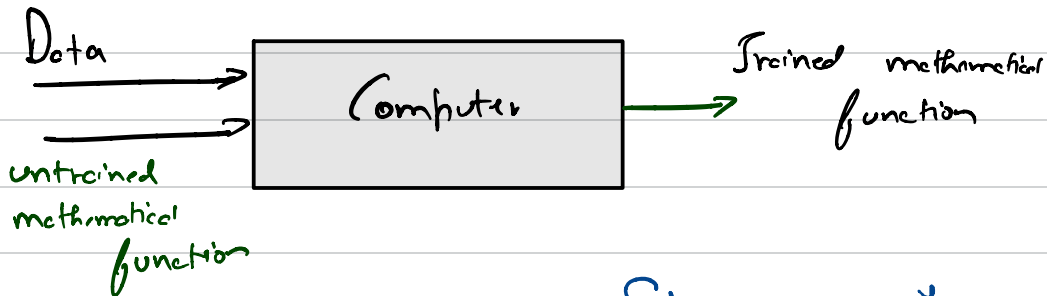
Arthur Samuel - ML provides a way to learn, without being explicitly programmed.

If word in email in ['lottery', 'prize', 'nigerian prince']:

email = 'spam'

else:

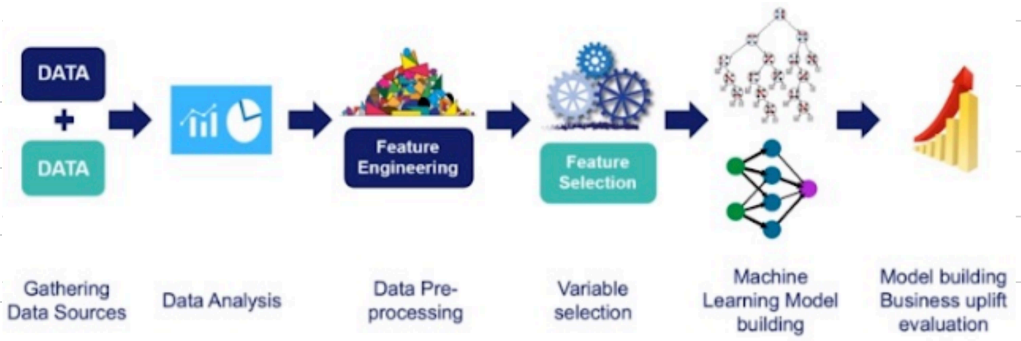
email = 'not-spam'



$$\text{Salary} = w \times \text{YOE}$$

YOE	SALARY
1	10
2	20
3	30
4	40
5	50
6	60

$$\text{Salary} = 10 \times \text{YOE}$$



Weight, Height, BMI, length of hair, skin-color | Diabetic

YOE	SALARY
1	30
2	40
3	50
4	60
5	70
6	80

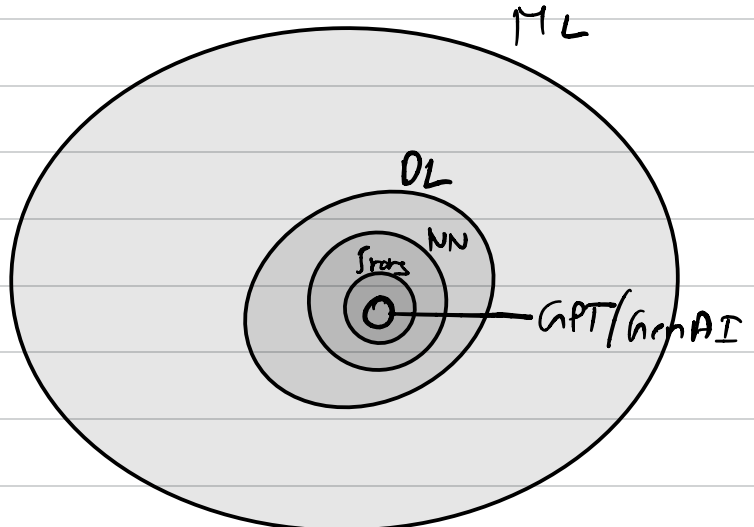
untrained mathematical function

$$y = wx + c$$

$$y = 20x + 10$$

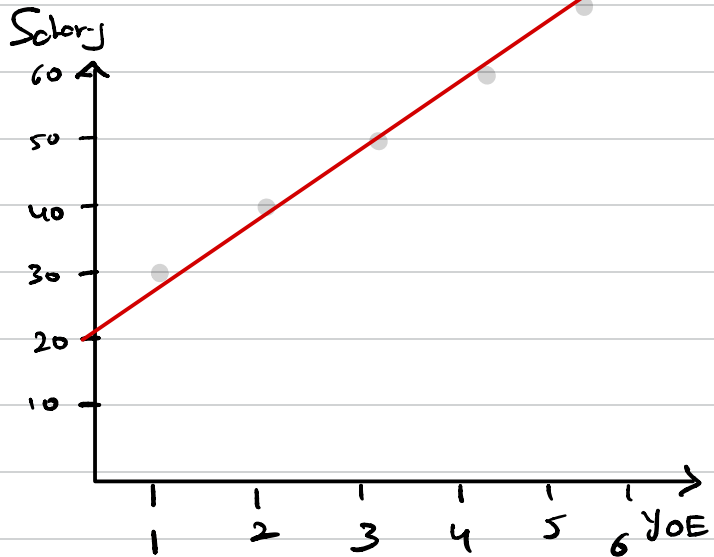
$$w = 20$$

$$c = 10$$



Linear Regression

YOE	SALARY
1	30
2	40
3	50
4	60
5	70
6	80

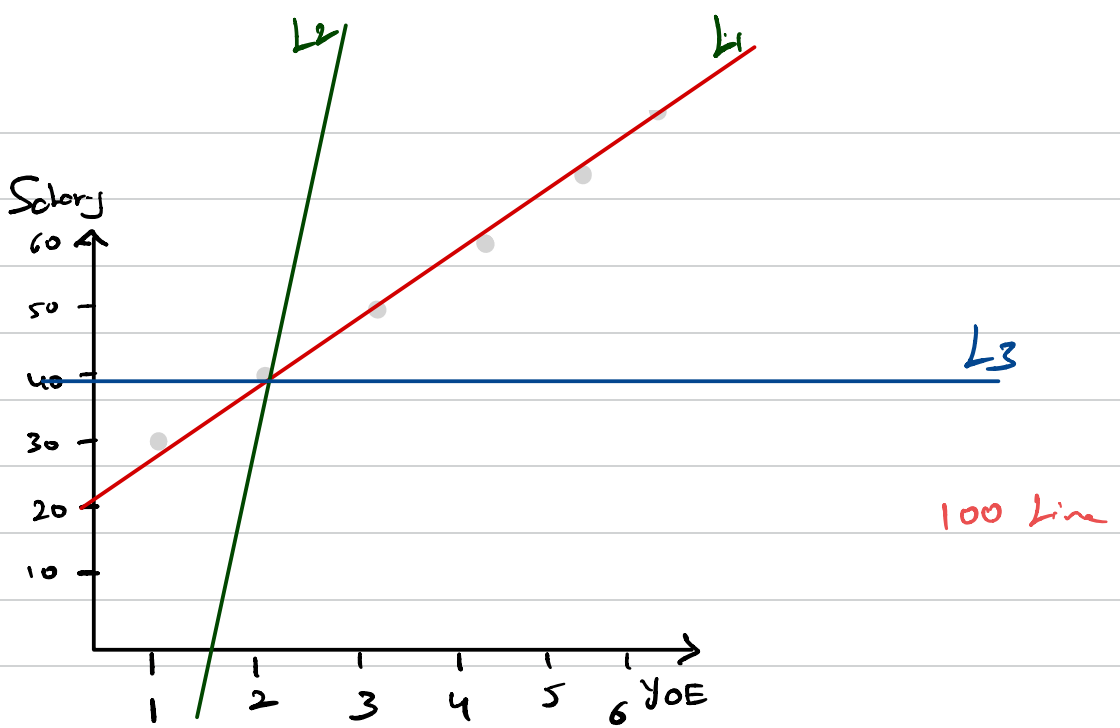


Mathematical function that can capture Pattern b/w input and output.

	f_1	f_2	f_3	f_4	f_5	f_6	y
x_1							
x_2							
x_3							
x_4							
x_5							
x_6							
x_7							
x_8							

Lot of money

$$\text{prediction} = f(\quad)$$

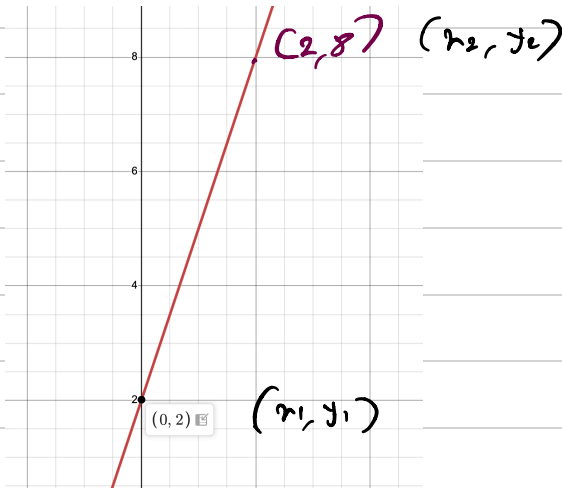


NN $\rightarrow$ $\frac{10 - 15}{10Ks}$ — Lett's of R

$$y = mn + c$$

$\downarrow$ slope $\downarrow$ Intercept

$\hookrightarrow \frac{\Delta y}{\Delta x} = \frac{dy}{dx} = \frac{\text{change in } y}{\text{change in } x}$



$$\frac{y_2 - y_1}{x_2 - x_1}$$

$$\frac{8 - 2}{2 - 0} = \frac{6}{2} = 3$$

$$\frac{2 - 8}{0 - 2} = \frac{-6}{-2} = 3$$

$$y = wx + c \rightarrow 0$$

Bedrooms	Actual rent (₹k)	$\hat{y}(w=10)$	$y_i - \hat{y}$	$(y_i - \hat{y})^2$
1	12	10	2	4
1	8	10	-2	4
2	22	20	2	4
2	18	20	-2	4
3	33	30	3	9
3	27	30	-3	9
4	46	40	6	36
4	34	40	-6	36
5	55	50	5	25
5	45	50	-5	25

$$MSE = \frac{1}{N} \sum_{i=1}^n (y_i - \hat{y})^2 = 15.6$$

$$\frac{1}{N} \sum_{i=1}^n |y_i - \hat{y}|$$

$$RMSE = \sqrt{MSE}$$

Why would we care about RMSE?

$$y = w_1 \times \text{balcony}$$

w_1
of balcon.

w_2
Sq. Ft

w_3

Bedrooms	Actual rent (₹k)
1	12
1	8
2	22
2	18
3	33
3	27
4	46
4	34
5	55
5	45

$$+ w_2 \times \text{Sq. Ft} \\ + w_3 \times \text{Bedrooms} \\ + C$$

$$w_1 = [-10, 10], 0.1$$

$$-10, -9.9, -9.8 \dots$$

$$100$$

$$100 +$$

$$0 - 10$$

$$= 200$$

$$\lambda = \lambda_{max}$$

$$\lambda_{max} - \lambda_{min}$$

Normalizing Data

$$y = w_1 x_1 + w_2 x_2 + c$$

$$w_1 = [-10, 10], 0.1 = 200$$

$$w_2 = [-10, 10], 0.1 = 200$$

$$c = [-10, 10], 0.1 = 200$$

For w_1 in range(-10, 10, 0.1): ~~200~~

for w_2 in range(-10, 10, 0.1): ~~200~~

for c in range(-10, 10, 0.1): ~~200~~

loss

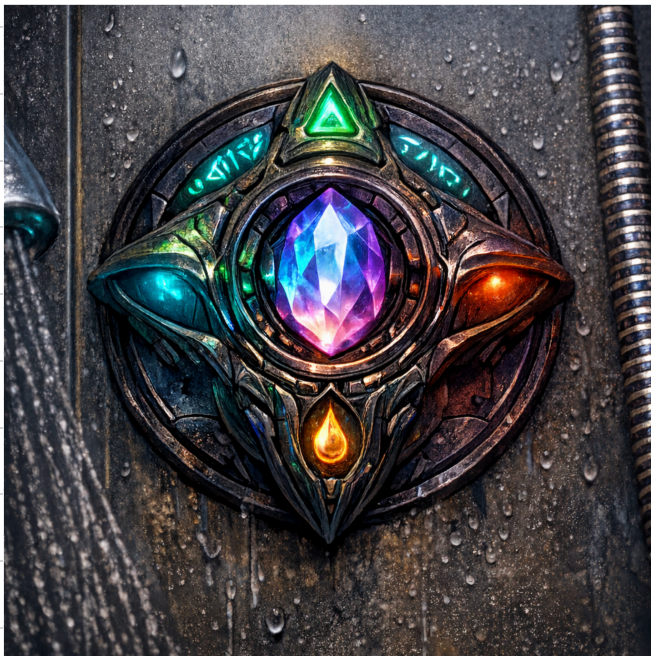
if loss < min_loss;

best_pair = [w1, w2, c]

$$(200^3) \times 10^{-6}$$

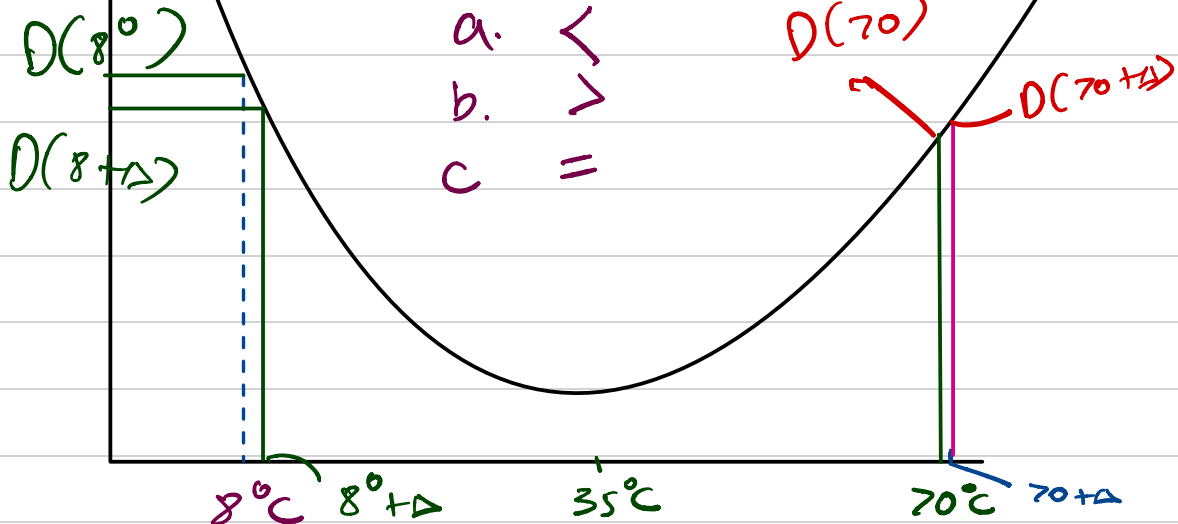
Core ML Workflow

1. **Define:** pick your **features (X)** and **target (y)**, and a baseline.
2. **Prepare data:** collect, clean, and do minimal preprocessing.
3. **Split:** train / validation / test (avoid leakage).
4. **Model:** choose a model family (e.g., linear, tree).
5. **Loss + metric:**
 - **Loss** = what you optimize during training
 - **Metric** = what you report/compare (e.g., MAE/RMSE, F1/AUC)
6. **Train:** fit on train set by minimizing loss. You nudge the random parameter, 'such that loss decreases
7. **Evaluate + iterate:** check metric on val/test, fix errors, tune, repeat.



Dis. Form

$$D(x) \square \triangleright D(x+\Delta)$$

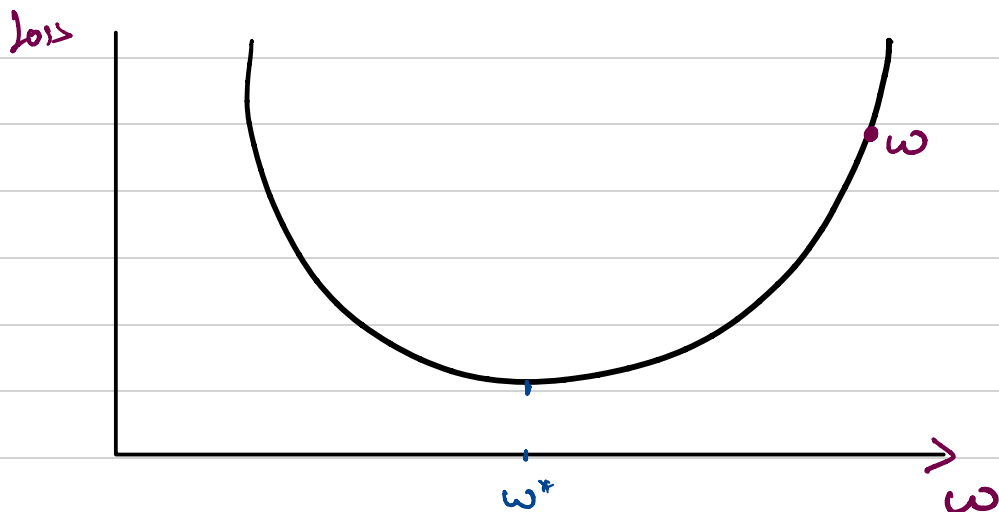


$$\frac{D(70+\Delta) - D(70)}{70+\Delta - 70}$$

y_2 y_1
 x_2 x_1

If my slope is positive: go left!
If my slope is negative: go right!!

$$\frac{D(8+\Delta) - D(8)}{8+\Delta - 8} = \Delta e$$



$$\frac{\text{Loss}(w + \Delta) - \text{Loss}(w)}{w + \Delta - w}$$

Eq 19

η = step size

$$w_{\text{new}} = w_{\text{old}} - \eta \times \frac{\partial \text{Loss}}{\partial w}$$

$$w_{\text{new}} = w_{\text{old}} - \eta \times \left(\frac{\partial \text{Loss}}{\partial w} \right)$$

Accuracy Metric

$y_{pred} = [0 \quad 1 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0]$
 $y_{actual} = [0 \quad 1 \quad 0 \quad 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0]$

$$\text{Accuracy} = \frac{\# \text{ Correct}}{\# \text{ Rows}} = \frac{7}{9}$$

100 errors → 90 errors N.S Acc: $\frac{90}{100}$
 10 errors S

model predicts 100 errors = 90%
 N.S

Acc metre:

① horrible for
class imbalance.

My Boss when finds out my Model is 90% accurate!

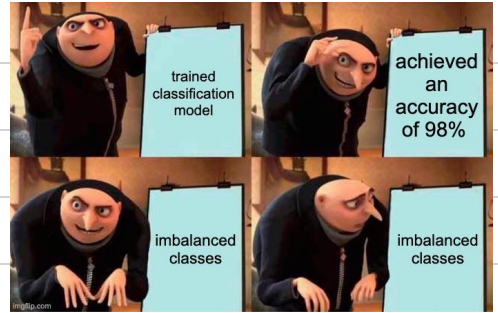


**Me: Trained my Model
on an imbalanced Dataset
with 90% of one class!**



Spam → (+)ve Class

Not-Spam → (-)ve Class



Actual

Predicted

Spam

Spam

True Positive

Spam

Not-Spam

False Negative

Not-Spam

Spam

False Positive

Not-Spam

Not-Spam

True Negative

CONFUSION MATRIX



Predictions

		Not Slam	Slam
Actuals	Not Slam	TN	FP
	Slam	FN	TP

Q1.

400 Emails

Actual $\left(\begin{array}{l} 360 - \text{N.S} \\ 40 - \text{S} \end{array} \right.$

Model $\rightarrow 400 - \text{W.S.}$

$$TN + FN = 400$$

Not spam

Spam

Not spam	Spam
TN 360	FP
FN 40	TP

$$PR = \frac{TP}{TP + FP} = 0$$

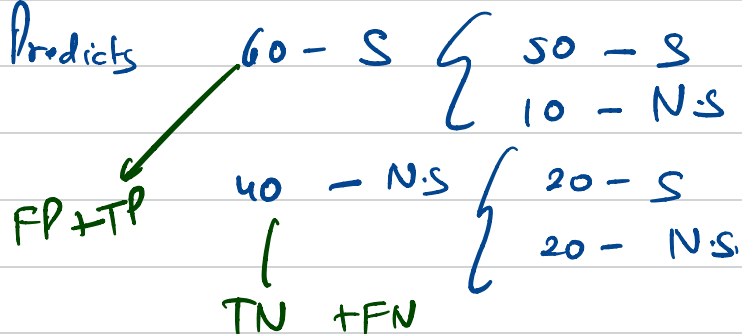
$$R = \frac{TP}{TP + FN}$$

Q2.

100 emails in total

$$N.S = 30, S = 70$$

Predicts



Not spam

Spam

Not spam	Spam
TN 20	FP 10
FN 20	TP 50

Q.3

Total Transactions: 10000

Fraud: 200

Not Fraud: 9800

Not Spam		Spam	
Not Spam		Spam	
TN	FP	FN	TP
9660	140	40	160

M → Fraud 300 $\left(\begin{array}{l} 160 - F \\ 140 - NF \end{array} \right)$

N.Fraud 9700 $\left(\begin{array}{l} 40 - F \\ 9660 - NF \end{array} \right)$

RP + TP

Q3.

500 - Patients

C = 80 , NC = 420

Model C = 60 $\left(\begin{array}{l} 50 - C \\ 10 - N.C \end{array} \right)$

N.C = 440 $\left(\begin{array}{l} 30 - C \\ 410 - N.C \end{array} \right)$

	Not spam	Spam
Not spam	TN	FP
Spam	FN	TP

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{FN + TP}$$

Case 1:

1. You incorrect received an email for donation from nigerian prince.
2. Gmail incorrectly classified a job offer letter from your dream company in your spam and you missed it.

Case 2:

1. Healthy patient was identified as covid positive
2. Covid patient was identified as covid negative.

